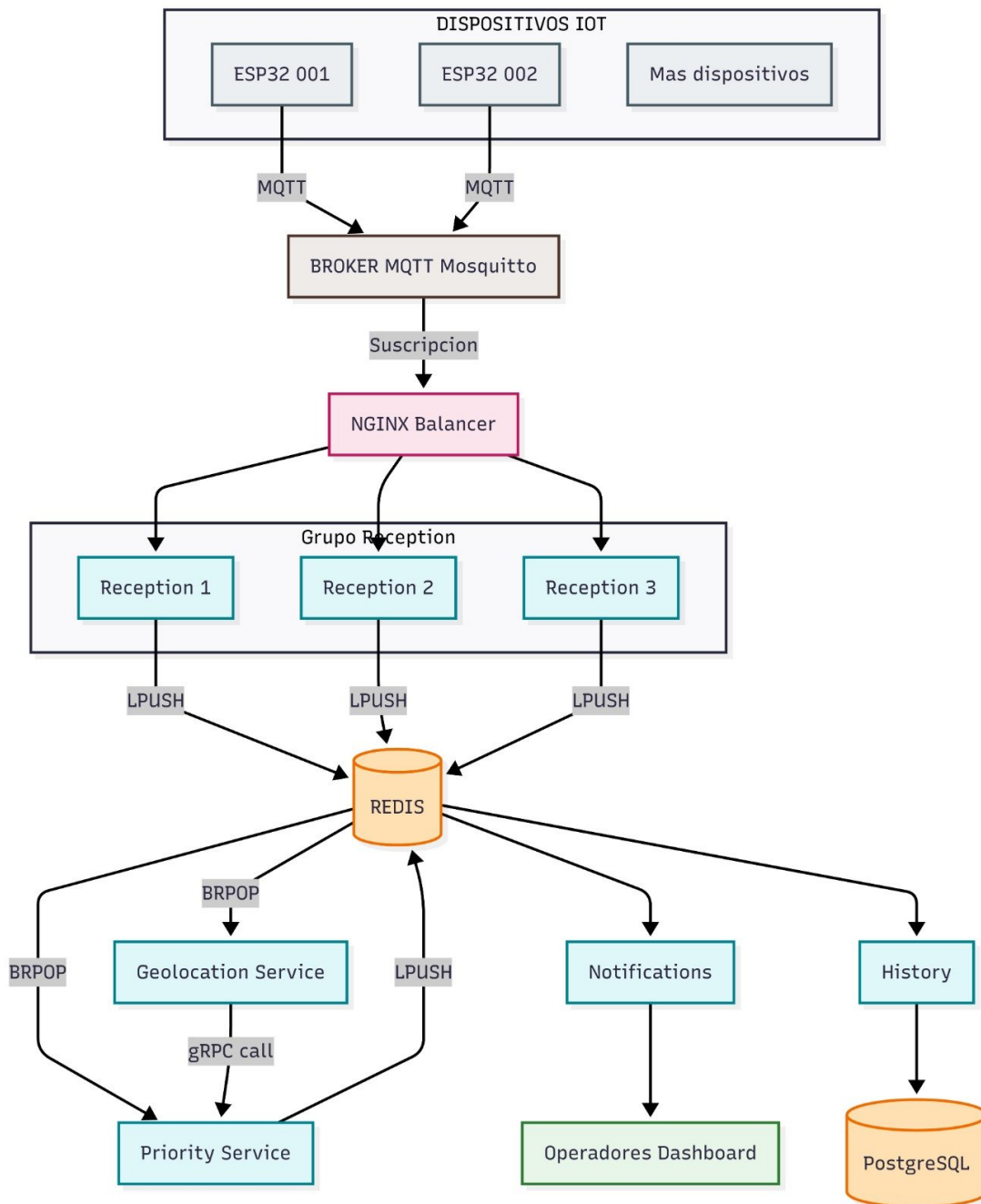


Documentación de Arquitectura: Sistema C5 Campus

1. Visión General

El sistema C5 Campus es una arquitectura orientada a eventos diseñada para procesar alertas de pánico provenientes de dispositivos IoT en tiempo real, garantizando escalabilidad y alta disponibilidad.

2. Diagrama de Arquitectura



3. Componentes del Sistema

Componente	Descripción
Dispositivos IoT	Nodos ESP32 que envían señales de pánico mediante el protocolo MQTT .
Mosquitto Broker	Servidor de mensajería que centraliza y distribuye los mensajes del topic <code>c5/alerts/#</code> .
NGINX (Balancer)	Balanceador de carga que distribuye el tráfico hacia las instancias de recepción utilizando <code>least_conn</code> .
Grupo Reception	Microservicios que consumen alertas y las registran en la cola principal de Redis.
Redis	Broker de colas y almacenamiento en memoria. Gestiona el ciclo de vida de la alerta (<code>alerts</code> → <code>geo_processed</code> → <code>notify</code> → <code>history</code>).
Geolocation Service	Servicio encargado de enriquecer la alerta con datos geoespaciales mediante una API REST.
Priority Service	Servicio crítico (gRPC) que clasifica la severidad de la alerta (crítica/alta/media).
PostgreSQL	Base de datos relacional para el almacenamiento persistente de auditoría e historial.

4. Flujo de Datos (Workflow)

1. **Ingesta:** Los dispositivos IoT publican alertas en el Broker MQTT.
2. **Recepción:** NGINX balancea las peticiones hacia el grupo Reception, que realiza un LPUSH en la cola de Redis.
3. **Procesamiento:** - El servicio de **Geolocation** y **Priority** consumen (BRPOP) las alertas en paralelo.
 - El servicio de **Priority** realiza un llamado gRPC al servicio de geolocalización para consolidar los datos.
4. **Notificación y Almacenamiento:** Una vez clasificada, la alerta se redirige a las colas de notificación y de historial.
5. **Consumo:** El Dashboard consume notificaciones vía WebSockets, mientras que el historial se persiste en PostgreSQL.

5. Consideraciones Técnicas

- **Tolerancia a fallos:** Se implementa una cola específica (`queue:notify_pending`) para asegurar que ninguna alerta se pierda.
- **Escalabilidad:** El uso de Redis como bus de mensajes permite añadir instancias de *Reception* o *Workers* de procesamiento de forma horizontal sin modificar la lógica central.

- **Comunicación:** Se utiliza una combinación híbrida de **MQTT** para telemetría, **REST** para APIs de consulta, **gRPC** para comunicación interna de alta eficiencia y **WebSockets** para tiempo real.